FridgeSense — HackCamp 2026 Build Plan

Nov 7, 2026 · @Someone

Overview

We are building a sensor that sits in a shared UBC residence fridge, streams air quality readings into Databricks, and uses an ML model to tell RAs which fridges are going bad, through a website they log into.

Problem: Shared floor fridges fill up with forgotten food. It rots, the whole floor complains, and RAs end up policing it with sniff tests and cleanout notices.

Users:

- **RAs:** need to know which fridge needs a cleanout before it becomes a crisis
- **Residents:** want a clean fridge and fair warning before their food gets thrown out

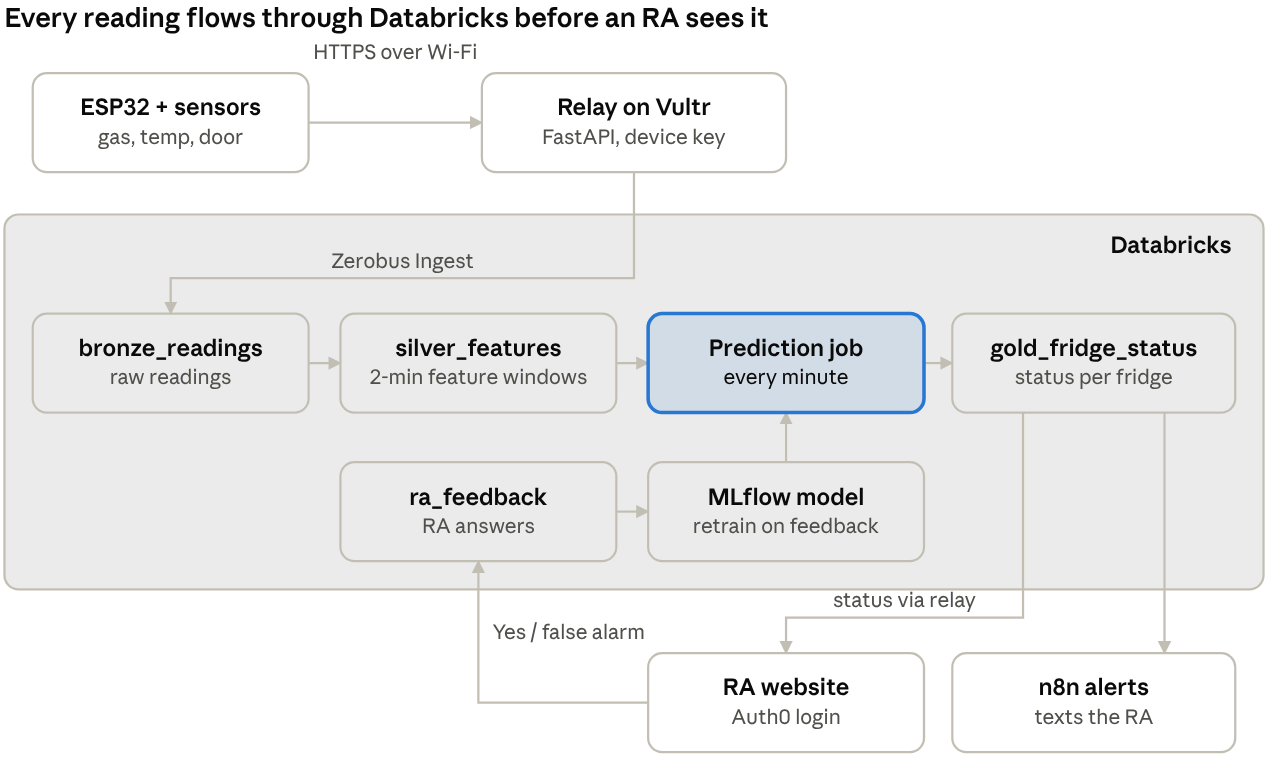
One-line pitch: A $20 sensor and an ML model that spots rotting food in shared dorm fridges, so RAs act before the floor smells it.

Target prizes:

Prize	What we need
Overall / Beginner Overall	Automatic
Best Hardware	Opt in, working sensor in the demo
Community Impact	Opt in, impact statement on Devpost
Best Use of Databricks	Opt in, Databricks used across ingest, ML and serving
Best use of n8n	Opt in, n8n sends RA alerts
Best Use of Vultr	Opt in, relay and website hosted on Vultr
Best Use of Auth0	Opt in, RA login
Best Domain Name (GoDaddy)	Register a clever domain

Architecture

The ESP32 sends readings over HTTPS to a relay on Vultr, which pushes them into Databricks; the model's output reaches RAs through the website and n8n alerts.



FridgeSense architecture · sensor to RA

RA feedback flows back into Databricks so the model retrains and improves over time.

Hardware

The device is an ESP32 with an air quality sensor, a temperature/humidity sensor and a door switch, sending one reading every 5 to 10 seconds over Wi-Fi.

Open question: exact air quality sensor model (check the label on the board).

Part	Purpose	Notes
ESP32	Reads sensors, sends data over Wi-Fi	Use a phone hotspot; eduroam and ubcvisitor will not work
Air quality sensor	Detects gases from rotting food	See wiring notes below

Part	Purpose	Notes
Temperature + humidity (DHT22 or similar)	Food safety, and corrects gas readings	Skip if the air quality board already has it (ENS160+AHT21, BME680)
Door switch (reed switch) or light sensor	Tells the model when the door opened	Light sensor works too: inside a closed box it reads dark
Power bank + USB cable	Runs on the judging table	Bring it charged
Small box or cooler	Our demo "fridge"	Same box for training data and the demo

Wiring by sensor type:

- **MQ-135 (analog):** runs on 5V and outputs up to 5V; the ESP32 reads max 3.3V, so use a voltage divider into an ADC pin. Needs several minutes of warm-up.
- **SGP30, CCS811, ENS160, BME680 (digital I2C):** SDA to GPIO 21, SCL to GPIO 22, 3.3V and GND. Read TVOC (or gas resistance on BME680).

ESP32 firmware tasks:

- [ ] Connect to Wi-Fi (phone hotspot)
- [ ] Sync time with NTP on boot
- [ ] Read all sensors every 5 to 10 seconds
- [ ] HTTPS POST JSON to the relay: `{device_id, timestamp, gas, temp, humidity, door_open}` with a device API key header
- [ ] Buffer readings in memory if Wi-Fi drops; resend on reconnect
- [ ] Serial print every reading for debugging

Relay and ingestion

A small always-on FastAPI server on Vultr receives readings from the ESP32 and pushes them into Databricks with Zerobus Ingest, so the system works from anywhere, not just from a laptop.

Relay endpoints:

- `POST /readings`: checks the device API key, forwards the reading to Zerobus
- `GET /status`: latest status per fridge, read from Databricks (used by the website)

- `GET /health`: for checking the server is up

Zerobus: REST insert into the `bronze_readings` Delta table. The relay holds the Databricks OAuth credentials (service principal) and refreshes the token; the ESP32 never sees them.

Fallback if Zerobus is not available on Free Edition: the relay writes batches of readings into the same table with the Databricks SQL Statement Execution API through the free SQL warehouse.

- [ ] Sign up for Databricks Free Edition; check Zerobus availability (timebox 30 min)
- [ ] Create a Vultr server, deploy the FastAPI relay with HTTPS
- [ ] Fake-data script that POSTs realistic readings to the relay (build first, unblocks everyone)
- [ ] Relay forwards to Zerobus, or to the SQL API fallback

Databricks data layer

Data flows through three tables (bronze, silver, gold): raw readings, then 2-minute feature windows, then the latest status per fridge.

Table	Contents	Written by
`bronze_readings`	device_id, timestamp, gas, temp, humidity, door_open	Zerobus (from the relay)
`silver_features`	device_id, window_end, gas_mean, gas_max, gas_slope, gas_std, temp_mean, humidity_mean, minutes_since_door_open	Lakeflow pipeline
`gold_fridge_status`	device_id, status, confidence, last_updated	Prediction job (every minute)
`ra_feedback`	device_id, alert_time, ra_says_rotten (yes/no)	Website feedback buttons
`fridges`	device_id, building, floor, ra_email	Set up once by hand

Free Edition limits to respect: one active pipeline per type, one SQL warehouse (2X-Small), a few model serving endpoints, no GPU. Our design fits inside these.

- [ ] Create the catalog, schema and tables
- [ ] Lakeflow pipeline: bronze to silver (clean readings, compute window features)
- [ ] Fill `fridges` with the demo fridge plus simulated ones

ML model

The model classifies each 2-minute window by the shape of the signal over time, not a simple threshold, so strong-but-fine food and door openings do not trigger false alarms.

Label	What the sensors show	RA sees
Normal	Flat, stable baseline	Fresh
Door opened	Short spike in gas and temperature, back to normal in minutes	Fresh
Strong food added	Sudden step up, then flat	Fresh
Spoiling	Slow, steady rise in gas	Watch / Going bad
Fridge failing	Temperature climbing steadily	Going bad

Data collection protocol (same sensor, same box as the demo; let the sensor warm up first):

1. Empty closed box: 20 min (normal)
2. Open and close the lid every 1 to 2 min: 15 min (door)
3. Fresh garlic, onion or cheese: 15 min each (strong food)
4. Something already going bad (old banana peel, food left out, warm milk): 20+ min (spoiling)
5. Warm the box, e.g. leave the lid off near a laptop: 15 min (fridge failing)

Record the start and end time of each run so the windows can be labeled.

Training (Databricks notebook):

- [ ] Load labeled windows from `silver_features`
- [ ] Train 3 models (Random Forest, gradient boosting, logistic regression) with scikit-learn

- [ ] Track each run in MLflow (accuracy, confusion matrix, feature importance)
- [ ] Register the best model; create a model serving endpoint
- [ ] Save a local copy of the model for the backup demo

Live predictions: a Databricks job every minute applies the model to the newest windows and writes `gold_fridge_status`.

Feedback loop (creative Databricks feature): RAs tap "Yes, it was rotten" or "False alarm" on each alert. Answers go to `ra_feedback`; a scheduled job retrains, and MLflow shows accuracy across model versions.

Be honest in the pitch: real spoilage takes days, so this is a proof of concept trained on a small dataset that improves as more fridges contribute.

RA website

RAs log in with Auth0 and see only their own building's fridges as Fresh, Watch or Going bad; the server reads from Databricks so no credentials reach the browser.

Pages:

- **Login** (Auth0)
- **Fridge list:** each fridge with status colour, confidence, last updated, and "sensor offline" if no reading for 2 min
- **Fridge detail:** gas and temperature chart over time, recent alerts, "Was this right?" buttons
- **Public demo page** (no login): live status of the demo fridge, so judges can watch on their own phone

Alerts (n8n): when a fridge turns Going bad, n8n texts or emails its RA with a ready-to-post cleanout notice.

Stretch: translate the cleanout notice into residents' languages with Databricks AI Functions (`ai_query`).

- [ ] Figma wireframes of the 3 pages (needed for design prizes)
- [ ] Website shell with Auth0 login, showing dummy statuses
- [ ] Connect to Databricks gold table through the relay's `GET /status`
- [ ] Detail page chart and feedback buttons
- [ ] n8n workflow: watch status, send RA alert

- [ ] Register the domain (GoDaddy Registry)

Team roles and build order

Four roles work in parallel from the start, because the fake-data script lets everyone build before the hardware is ready.

Role	Owns
Hardware	ESP32 + sensors, firmware, labeled data collection
Data / ML	Databricks tables, pipeline, model training, prediction job, feedback loop
Backend	Vultr relay, Zerobus, SQL API, Auth0, n8n
Frontend / design / pitch	Figma, website, RA interviews, Devpost, pitch

Open question: judging time (check Discord) and whether we are in the beginner stream.

Build order (Saturday afternoon to Sunday morning):

1. **By 4pm:** fake-data script, Databricks signup + Zerobus check, Vultr server, Figma wireframes, RA interviews started
2. **By 7pm:** relay to Databricks working end to end with fake data; website login with dummy statuses
3. **By 10pm:** real sensor data flowing; labeled data collection done
4. **By 1am:** model trained and registered; prediction job writing `gold_fridge_status`; website shows real statuses
5. **By 4am:** n8n alerts, feedback buttons, simulated fridges, polish
6. **Feature freeze, then:** Devpost write-up, slides, rehearse the demo 3 times, sleep

Demo, pitch and Q&A

The demo proves three things live in under 90 seconds: the hardware works, the model is smarter than a threshold, and the RA gets alerted anywhere.

Live demo script:

1. Hand a judge a phone with the public demo page open
2. Open the box lid: page updates within seconds, classified as "door opened", still Fresh

3. Put fresh garlic in: classified "strong food", still Fresh (not a false alarm)
4. Replay recorded spoilage data (real spoilage takes days): status turns Going bad, RA phone buzzes with the n8n alert
5. Show Databricks: MLflow model comparison and accuracy improving from RA feedback

4-minute pitch (rubric: Problem Research, Technical, Design, Innovation, Pitch):

Time	Content
0:00 to 0:40	The problem, opened with a real quote from a res student or RA
0:40 to 2:10	Live demo
2:10 to 3:10	How it works: sensor, Databricks pipeline, ML model, RA website
3:10 to 3:40	Design and accessibility: Figma, simple 3-colour status
3:40 to 4:00	Impact: every shared fridge in UBC residences, about $20 in parts each

Q&A prep:

- **Is this surveillance?** It monitors the fridge, not people: no cameras, no names.
- **What about smelly but fine food?** The model looks at signal shape over time; we show it live with garlic.
- **How accurate is it?** Give the real test accuracy from MLflow and say it is a small proof-of-concept dataset.
- **Why Databricks?** One fridge is a few sensors; every residence fridge streaming all year is a real dataset, and Databricks handles ingest, training, serving and retraining in one place.
- **What if Wi-Fi drops?** The ESP32 buffers readings and resends; the dashboard flags offline sensors.
- **Deployment?** Cellular (LTE-M) or LoRaWAN modules so sensors do not depend on building Wi-Fi.

Problem research and Devpost checklist

Problem Research is a full rubric category, and the venue is full of UBC residence students, so collect real quotes this afternoon.

Ask 5 or more res students (and any RAs):

1. How often does your shared fridge get gross? Who deals with it?
2. Have you had food thrown out, or found something rotten?
3. (RAs) How much time do fridge complaints and cleanouts take?
4. Would an alert before it smells change what you do?

Write down exact quotes, names (with permission) and residence.

Devpost checklist:

- [ ] Problem statement and solution description
- [ ] Impact statement (Community Impact)
- [ ] Figma link (design prizes)
- [ ] How we used Databricks, Vultr, Auth0, n8n (one short paragraph each)
- [ ] Photos of the hardware and a screenshot of the dashboard
- [ ] Opt into every prize we qualify for (check if there is a cap on opt-ins)
- [ ] Domain name listed

Risks and fallbacks

Every live part of the demo has a backup, so nothing at the judging table depends on a single service working.

Risk	Fallback
Zerobus not available on Free Edition	SQL Statement Execution API into the same table
Databricks slow or endpoint not ready at judging	Local copy of the model in the relay; website reads from the relay
Sensor breaks or gives noisy data	Replay script of recorded readings; spare sensor from the hardware table
Venue Wi-Fi blocks the ESP32	Phone hotspot (plan A anyway)

Risk	Fallback
Not enough labeled data	Fewer classes (Fresh vs Going bad); be honest about dataset size
Running out of time	Cut stretch goals first: translation, simulated fleet, retraining
Laptop battery dies	Charger plus power bank for the ESP32